Puppet Base Course

Day 1

Fundamentals, Infrastructure & The Puppet Language

Day 1 — Agenda

Overview

- What is configuration management and why does it matter?
- Introduction to Puppet — the open-source project
- Advantages of the declarative approach
- Puppet vs. other configuration management tools

Puppet Infrastructure

- AIO (All-in-One) packages — what's included
- Installing and configuring the Puppet server
- Installing and configuring Puppet agents on Linux
- Managing security certificates (CSR, signing, revocation)

The Puppet Language — Foundations

- Purpose and function of the Puppet language
- Variables, data types, and arrays
- Defining resources: `file`, `package`, `service`, `user`, `group`, `exec`
- Resource ordering: `before`, `require`, `notify`, `subscribe`
- Conditional configuration: `if`, `unless`, `case`, `selector`
- Grouping resources into classes
- User-defined resources with `define`

Part 1 — Overview

What is Configuration Management?

The problem it solves:

- Servers accumulate manual changes over time — no two look alike
- A wiki page of installation steps is always out of date
- "It works on my machine" is not a deployment strategy
- Rebuilding a node from scratch takes days, not minutes

> This gradual divergence is called **configuration drift**. Configuration management tools eliminate it.

The Configuration Drift Problem

Without CM	With Puppet
Manual SSH → run commands	Declare desired state in code
Undocumented, unrepeatable	Every change is version-controlled
Snowflake servers — all different	All nodes converge to the same state
Incident at 2am: nobody knows what's installed	`puppet resource package` shows truth
Rebuild = days of effort	Rebuild = run the agent once

What Tools Exist?

Tool	Approach	Model	Language
Puppet	Declarative	Agent/Server	Puppet DSL
Ansible	Imperative	Agentless (SSH)	YAML playbooks
Chef	Declarative/Imperative	Agent/Server	Ruby DSL
SaltStack	Declarative/Imperative	Agent/Server or Agentless	YAML/Jinja

> Puppet is the oldest and most battle-tested of these — used at scale in banks, telcos, and hyperscalers worldwide.

Introduction to Puppet

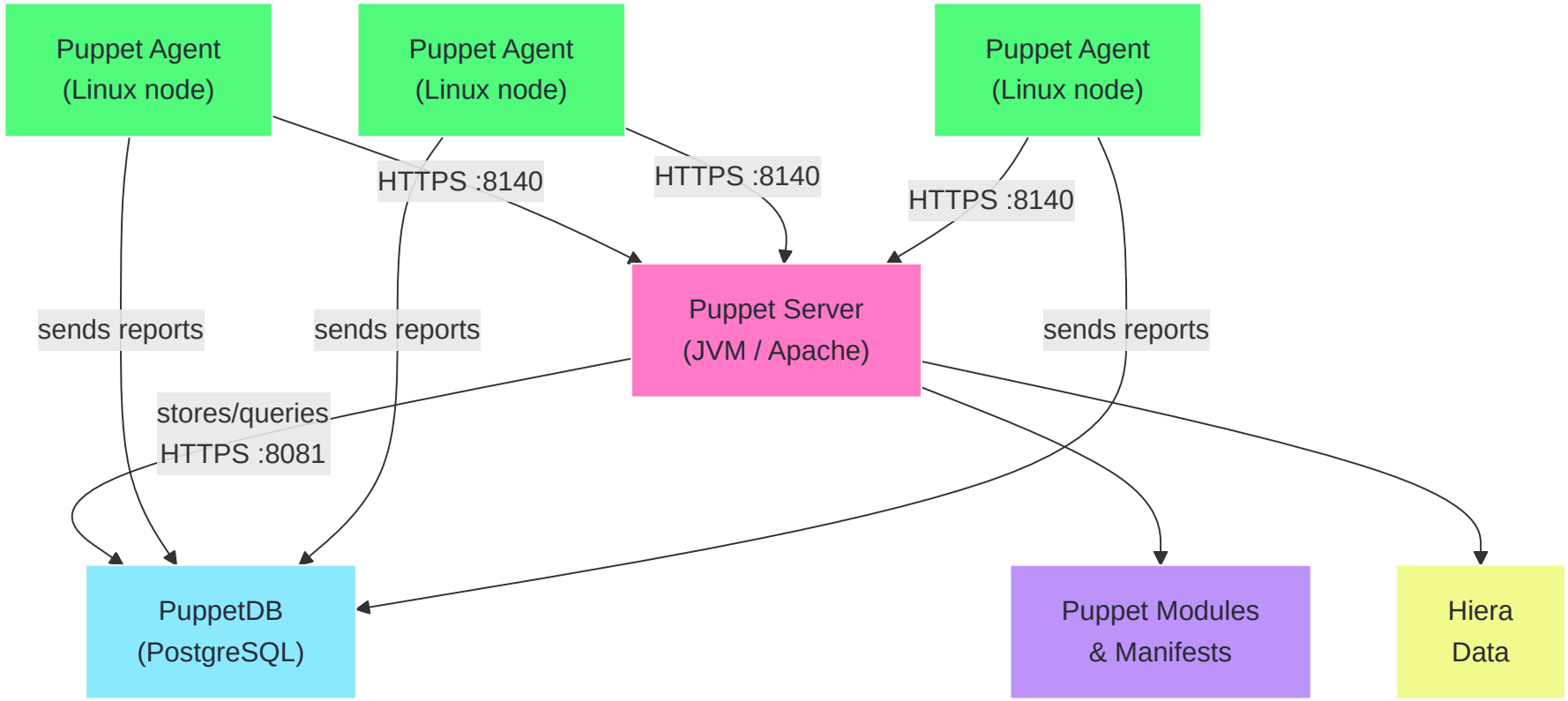
- **Born in 2005** — created by Luke Kanies, written in Ruby
- Acquired by Perforce in 2022 after years as an independent company
- Available as **open source** (`puppet-agent`, `puppetserver`) and commercial (**Puppet Enterprise**)
- Uses a **declarative domain-specific language (DSL)** — you describe *what* you want, not *how* to get there
- Manages nodes via a **catalog** — a compiled list of resources to apply

> "Automate the things that should always be the same."

Declarative vs. Imperative

	Declarative (Puppet)	Imperative (Shell scripts)
You describe	Desired end state	Steps to get there
Example	`ensure => installed`	`apt-get install -y nginx`
Idempotent	Always — built in	Only if you add guards
Drift detection	Automatic on every run	Manual comparison
Error on re-run	None	Error: "package already installed"

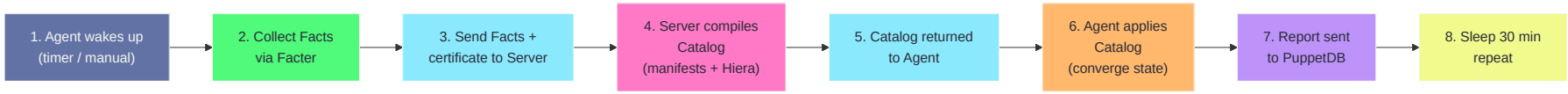
How Puppet Works — The Big Picture



Key Components

Component	Role
Puppet Agent	Runs on every managed node; enforces the catalog
Puppet Server	Compiles catalogs; serves files; central CA
PuppetDB	Stores facts, catalogs, and reports in PostgreSQL
Facter	Collects system facts (OS, IP, memory, …) on the agent
Hiera	Hierarchical key/value store for separating data from code
Puppet Forge	Community module registry — like Terraform Registry

The Agent Run Cycle



The Catalog

The **catalog** is a directed acyclic graph (DAG) of resources:

- Compiled by the server from manifests + Hiera data + facts
- Specific to **one node** — every node gets its own catalog
- Describes **every resource** that should exist on that node and in what state
- The agent **applies** the catalog — converging the actual state to desired state

```
Catalog for web01.example.com:
  Package[nginx]          ensure=installed
  File[/etc/nginx/nginx.conf]  content=<...>
  Service[nginx]          ensure=running, enable=true
```

Part 2 — Puppet Infrastructure

AIO (All-in-One) Packages

Puppet distributes everything in a single AIO package:

```
puppet-agent
├── puppet        ← the agent binary
├── facter        ← fact collection
├── hiera         ← data lookup
├── mcollect      ← orchestration (legacy)
├── pxp-agent     ← Puppet task runner
└── Ruby runtime  ← bundled Ruby, no system Ruby needed
```

- Installed to `/opt/puppetlabs/`
- Binaries linked to `/usr/local/bin/`
- **No dependency on system Ruby** — eliminates conflicts
- One package name everywhere: `puppet-agent`

Installing Puppet Server

Step 1 — Configure the Puppet platform repository

```
# RHEL / Rocky / AlmaLinux 8+
sudo rpm -Uvh https://yum.puppet.com/puppet8-release-el-8.noarch.rpm
sudo dnf install -y puppetserver

# Ubuntu 22.04
wget https://apt.puppet.com/puppet8-release-jammy.deb
sudo dpkg -i puppet8-release-jammy.deb
sudo apt-get update && sudo apt-get install -y puppetserver
```

Step 2 — Configure server memory

Edit `/etc/sysconfig/puppetserver` (RHEL) or `/etc/default/puppetserver` (Debian):

```
# Default is 2g — fine for up to ~20 agents
# Increase to 4g for 20–100 agents, 8g+ for 100+
JAVA_ARGS="-Xms2g -Xmx2g -Djruby.logger.class=com.puppetlabs.jruby_utils.jruby.Slf4jLogger"
```

Step 3 — Start and enable the service

```
sudo systemctl start puppetserver
sudo systemctl enable puppetserver
```

Step 4 — Verify

```
sudo systemctl status puppetserver

# Check server is listening on port 8140
sudo ss -tlnp | grep 8140

# View server logs
sudo journalctl -u puppetserver -f
```

> The Puppet Server's hostname defaults to `puppet`. Agents will connect to `puppet` by default. Set DNS or `/etc/hosts` accordingly.

Puppet Server Directory Layout

```
/etc/puppetlabs/
├── puppet/
│   ├── puppet.conf          ← main configuration file
│   ├── hiera.yaml           ← Hiera configuration
│   └── ssl/                 ← CA and certificates
│       ├── ca/              ← Certificate Authority data
│       ├── certs/           ← Signed agent certificates
│       ├── private_keys/    ← Server private keys
│       └── public_keys/     ← Public keys
└── code/
    └── environments/
        └── production/      ← Default Puppet environment
            ├── manifests/
            │   └── site.pp  ← Node classification entry point
            ├── modules/     ← Module path
            └── hiera.yaml
```

Installing the Puppet Agent on Linux

```
# RHEL / Rocky / AlmaLinux
sudo rpm -Uvh https://yum.puppet.com/puppet8-release-el-8.noarch.rpm
sudo dnf install -y puppet-agent

# Ubuntu 22.04
wget https://apt.puppet.com/puppet8-release-jammy.deb
sudo dpkg -i puppet8-release-jammy.deb
sudo apt-get update && sudo apt-get install -y puppet-agent
```

Configure the agent to point at the server:

```
# /etc/puppetlabs/puppet/puppet.conf
[main]
server = puppet.example.com
environment = production

[agent]
runinterval = 1800   # run every 30 minutes
```

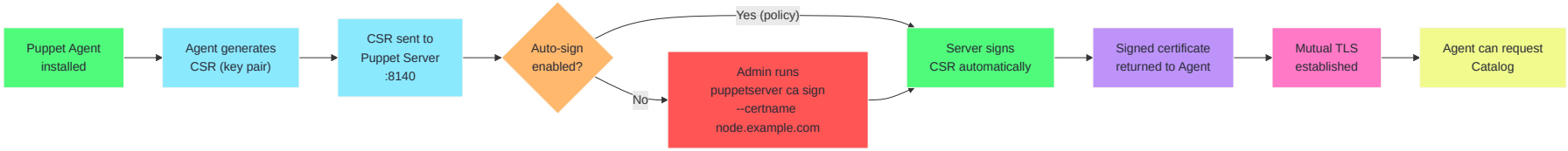
Starting the Agent and Triggering a Run

```
# Start the agent service (runs every 30 min by default)
sudo systemctl start puppet
sudo systemctl enable puppet

# Trigger an immediate run (useful during setup and testing)
sudo puppet agent --test

# Run in no-op (dry-run) mode — see what WOULD change, apply nothing
sudo puppet agent --test --noop
```

Managing Security Certificates



The Certificate Lifecycle — In Detail

1. First `puppet agent --test` generates a key pair and sends a **Certificate Signing Request (CSR)** to the server
2. The server receives the CSR and queues it for approval
3. Admin lists and signs pending requests:

```
# List pending CSRs
sudo puppetserver ca list

# Sign a specific agent
sudo puppetserver ca sign --certname agent01.example.com

# Sign all pending CSRs (use carefully in production!)
sudo puppetserver ca sign --all
```

Certificate Management — Day-to-Day Operations

```
# List all certificates (signed, requested, revoked)
sudo puppetserver ca list --all

# Revoke a certificate (decommission a node)
sudo puppetserver ca revoke --certname agent01.example.com

# Clean (delete) a certificate — agent can re-enroll
sudo puppetserver ca clean --certname agent01.example.com

# On the agent: remove local cert and re-request
sudo puppet ssl clean
sudo puppet agent --test  # re-generates CSR
```

Auto-signing (Use with Care)

For ephemeral environments (CI, cloud auto-scaling) you can configure **policy-based autosigning**:

```
# /etc/puppetlabs/puppet/autosign.conf
# Simple: allow any host in your domain (not recommended for production)
*.example.com

# Better: use a policy script
# autosign = /usr/local/bin/autosign-policy.sh
```

Approach	Use case	Risk
Manual signing	Production nodes	None — secure by default
Domain autosign	Internal dev/test	Low — trust your DNS
Policy script	Cloud auto-scale	Medium — validate token

Part 3 — The Puppet Language

The Puppet DSL — Purpose

- A **declarative domain-specific language** — not general-purpose programming
- Defines **resources** (things that should exist) and their **desired state**
- Puppet compiles it into a **catalog** and applies it
- Files end in `.pp` — "Puppet Program"
- The language is **idempotent by design** — Puppet only changes what needs changing

Your First Puppet Manifest

```
# /etc/puppetlabs/code/environments/production/manifests/site.pp

node 'web01.example.com' {
  package { 'nginx':
    ensure => installed,
  }

  file { '/var/www/html/index.html':
    ensure  => file,
    content => "<h1>Managed by Puppet</h1>\n",
    owner   => 'www-data',
    group   => 'www-data',
    mode    => '0644',
  }

  service { 'nginx':
    ensure => running,
    enable => true,
  }
}
```

Anatomy of a Resource Declaration

```
resource_type { 'resource_title':
  attribute1 => value1,
  attribute2 => value2,
}
```

Part	Meaning
`resource_type`	What kind of thing (file, package, service, …)
`'resource_title'`	Unique name — usually the path or package name
`attribute => value`	What state you want

> The combination of **type + title** must be unique per catalog.

Variables

Variables in Puppet start with `$` and are **immutable** (assigned once per scope):

```
$package_name  = 'nginx'
$config_dir    = '/etc/nginx'
$port          = 80
$is_production = true

package { $package_name:
  ensure => installed,
}

file { "${config_dir}/nginx.conf":
  ensure  => file,
  content => "worker_processes auto;\n",
}
```

> Strings with variables inside must use **double quotes**: `"${variable}"`.
> Single-quoted strings are literals: `'$not_interpolated'`.

Data Types

```
# String
$name = 'web01'

# Integer / Float
$port    = 8080
$timeout = 30.5

# Boolean
$enabled = true
$debug   = false

# Undef (null)
$optional = undef

# Array
$packages = ['nginx', 'vim', 'curl', 'htop']

# Hash
$user_config = {
  'shell' => '/bin/bash',
  'home'  => '/home/deploy',
}
```

Working with Arrays

```
$packages = ['nginx', 'vim', 'curl']

# Iterate with each (more on this in Day 2)
$packages.each |$pkg| {
  package { $pkg:
    ensure => installed,
  }
}

# Access by index (0-based)
$first = $packages[0]   # 'nginx'

# Array functions
$count = $packages.length     # 3
$has   = 'nginx' in $packages # true
```

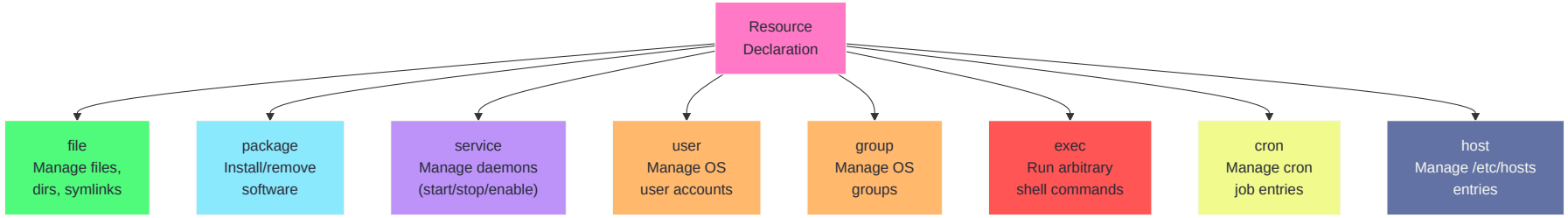
Working with Hashes

```
$vhosts = {
  'site1' => { 'port' => 80,  'root' => '/var/www/site1' },
  'site2' => { 'port' => 443, 'root' => '/var/www/site2' },
}

# Access hash values
$port = $vhosts['site1']['port']   # 80

# Iterate over a hash
$vhosts.each |$name, $config| {
  file { "/etc/nginx/sites-available/${name}":
    content => "root ${config['root']};\n",
  }
}
```

Core Resource Types



The `file` Resource

```
# Create a file
file { '/etc/motd':
  ensure  => file,
  content => "Welcome to ${facts['hostname']}\n",
  owner   => 'root',
  group   => 'root',
  mode    => '0644',
}

# Create a directory
file { '/opt/myapp':
  ensure => directory,
  owner  => 'deploy',
  mode   => '0755',
}

# Create a symlink
file { '/usr/local/bin/myapp':
  ensure => link,
  target => '/opt/myapp/bin/myapp',
}

# Serve a file from the module's files/ directory
file { '/etc/myapp/config':
  ensure => file,
  source => 'puppet:///modules/mymodule/config',
}
```

The `package` Resource

```
# Ensure installed (any version)
package { 'vim':
  ensure => installed,
}

# Ensure a specific version
package { 'nginx':
  ensure => '1.24.0',
}

# Ensure removed
package { 'telnet':
  ensure => absent,
}

# Ensure always on latest version (use with caution!)
package { 'curl':
  ensure => latest,
}

# Install multiple packages at once
package { ['htop', 'tmux', 'git', 'jq']:
  ensure => installed,
}
```

The `service` Resource

```
# Running and enabled (starts on boot)
service { 'nginx':
  ensure => running,
  enable => true,
}

# Stopped and disabled
service { 'postfix':
  ensure => stopped,
  enable => false,
}

# Pattern for package → config → service
package { 'nginx':
  ensure => installed,
}

file { '/etc/nginx/nginx.conf':
  ensure  => file,
  content => epp('mymodule/nginx.conf.epp'),
  require => Package['nginx'],
  notify  => Service['nginx'],
}

service { 'nginx':
  ensure  => running,
  enable  => true,
  require => Package['nginx'],
}
```

The `user` and `group` Resources

```
group { 'deploy':
  ensure => present,
  gid    => 1500,
}

user { 'deploy':
  ensure     => present,
  uid        => 1500,
  gid        => 'deploy',
  home       => '/home/deploy',
  shell      => '/bin/bash',
  managehome => true,
  require    => Group['deploy'],
}

# Add SSH authorized key
ssh_authorized_key { 'deploy@ci':
  user => 'deploy',
  type => 'ssh-rsa',
  key  => 'AAAAB3NzaC1yc2EA...',
}
```

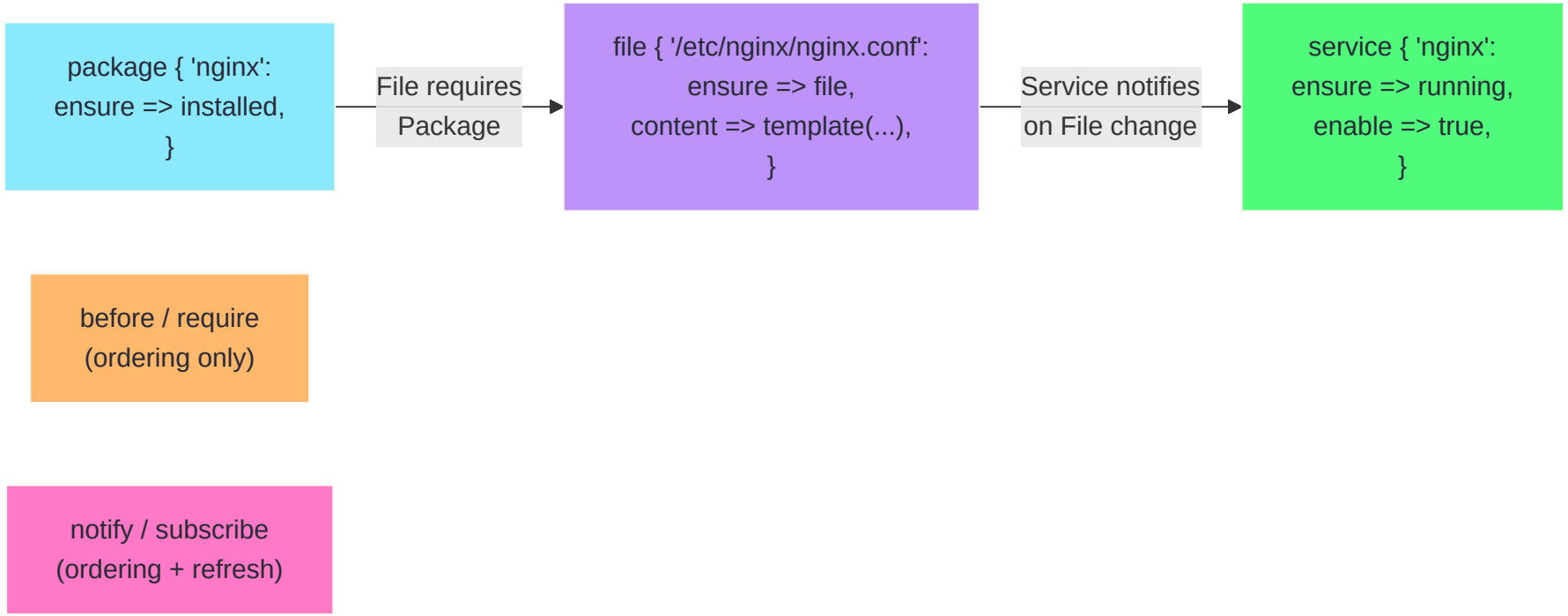
The `exec` Resource

```
exec { 'generate-ssl-cert':
  command => '/usr/bin/openssl req -x509 -newkey rsa:4096 -keyout /etc/ssl/key.pem -out /etc/ssl/cert.pem -days 365 -nodes',
  creates => '/etc/ssl/cert.pem',  # only runs if this file doesn't exist
  path    => ['/usr/bin', '/bin'],
}

exec { 'reload-nginx':
  command     => '/usr/sbin/service nginx reload',
  refreshonly => true,   # only runs when notified by another resource
  path        => ['/usr/sbin', '/usr/bin', '/bin'],
}
```

> **Use `exec` sparingly.** Prefer native resource types. Always add `creates`, `unless`, or `onlyif` to prevent re-execution.

Resource Ordering



Resource Ordering — The Four Metaparameters

```
# before: "I must be applied before this resource"
package { 'nginx': ensure => installed, before => File['/etc/nginx'] }

# require: "I need this resource applied first"
file { '/etc/nginx/nginx.conf': require => Package['nginx'] }

# notify: "Apply me first, and trigger a refresh on this resource"
file { '/etc/nginx/nginx.conf': notify => Service['nginx'] }

# subscribe: "Apply me after, and refresh if the other resource changes"
service { 'nginx': subscribe => File['/etc/nginx/nginx.conf'] }
```

Metaparameter	Direction	Triggers refresh?
`before` / `require`	Ordering only	No
`notify` / `subscribe`	Ordering + refresh	Yes

Resource Reference Syntax

When referring to a resource in ordering or relationships, use the **resource reference** capitalised:

```
Package['nginx']            # reference to a package resource
File['/etc/nginx']          # reference to a file resource
Service['nginx']            # reference to a service resource
Class['profile::nginx']     # reference to a class
```

> The title must exactly match the title in the resource declaration.

Conditional Configuration — `if` / `unless`

```
if $facts['os']['family'] == 'RedHat' {
  package { 'httpd':
    ensure => installed,
  }
} elsif $facts['os']['family'] == 'Debian' {
  package { 'apache2':
    ensure => installed,
  }
} else {
  fail("Unsupported OS family: ${facts['os']['family']}")
}

unless $facts['virtual'] == 'physical' {
  package { 'open-vm-tools':
    ensure => installed,
  }
}
```

Conditional Configuration — `case`

```
case $facts['os']['name'] {
  'Ubuntu', 'Debian': {
    $service_name = 'apache2'
    $conf_dir     = '/etc/apache2'
  }
  'RedHat', 'CentOS', 'Rocky': {
    $service_name = 'httpd'
    $conf_dir     = '/etc/httpd'
  }
  default: {
    fail("Module not supported on ${facts['os']['name']}")
  }
}

service { $service_name:
  ensure => running,
  enable => true,
}
```

Conditional Configuration — Selector

The selector is an **expression** (not a statement) — it evaluates to a value:

```
$package_name = $facts['os']['family'] ? {
  'RedHat' => 'httpd',
  'Debian' => 'apache2',
  default  => 'apache2',
}

$port = $environment ? {
  'production' => 443,
  'staging'    => 8443,
  default      => 8080,
}
```

> Use a selector when you want to assign based on a condition. Use `case` or `if` when you want to take different actions.

Classes

A **class** is a named block of Puppet code that can be included on any number of nodes:

```
class nginx {
  package { 'nginx':
    ensure => installed,
  }

  file { '/etc/nginx/nginx.conf':
    ensure  => file,
    source  => 'puppet:///modules/nginx/nginx.conf',
    require => Package['nginx'],
    notify  => Service['nginx'],
  }

  service { 'nginx':
    ensure  => running,
    enable  => true,
    require => Package['nginx'],
```

Declaring and Including Classes

```
# Include: safe to call multiple times — idempotent
include nginx
include nginx   # second call is silently ignored

# Class declaration: can pass parameters
class { 'nginx':
  worker_processes => 4,
  user             => 'www-data',
}

# In site.pp — assign class to a node
node 'web01.example.com' {
  include nginx
  include profile::base
}
```

> **Rule of thumb:** Use `include` to assign classes; use `class { }` syntax only when you need to override parameters and there is **no Hiera** in use.

Classes with Parameters

```
class nginx (
  Integer           $worker_processes = $facts['processors']['count'],
  String            $user             = 'www-data',
  Array[Integer]    $listen_ports     = [80, 443],
  Boolean           $enable_gzip      = true,
) {
  package { 'nginx':
    ensure => installed,
  }

  file { '/etc/nginx/nginx.conf':
    ensure  => file,
    content => epp('nginx/nginx.conf.epp', {
      worker_processes => $worker_processes,
      user             => $user,
      listen_ports     => $listen_ports,
      enable_gzip      => $enable_gzip,
    }),
  }
}
```

Class Inheritance

```
class apache {
  package { 'apache2': ensure => installed }
  service { 'apache2': ensure => running, enable => true }
}

class apache::ssl inherits apache {
  # Parent resources are available — override attributes
  Service['apache2'] {
    require +> File['/etc/ssl/certs/server.crt'],
  }

  package { 'libssl-dev': ensure => installed }

  file { '/etc/ssl/certs/server.crt':
    ensure => file,
    source => 'puppet:///modules/apache/server.crt',
  }
}
```

User-Defined Resources — `define`

A **defined type** is like a class that can be instantiated multiple times:

```
define vhost (
  String  $servername,
  String  $document_root = "/var/www/${title}",
  Integer $port          = 80,
) {
  file { "/etc/nginx/sites-available/${title}":
    ensure  => file,
    content => epp('nginx/vhost.conf.epp', {
      servername    => $servername,
      document_root => $document_root,
      port          => $port,
    }),
    notify  => Service['nginx'],
  }

  file { "/etc/nginx/sites-enabled/${title}":
    ensure => link,
    target => "/etc/nginx/sites-available/${title}",
```

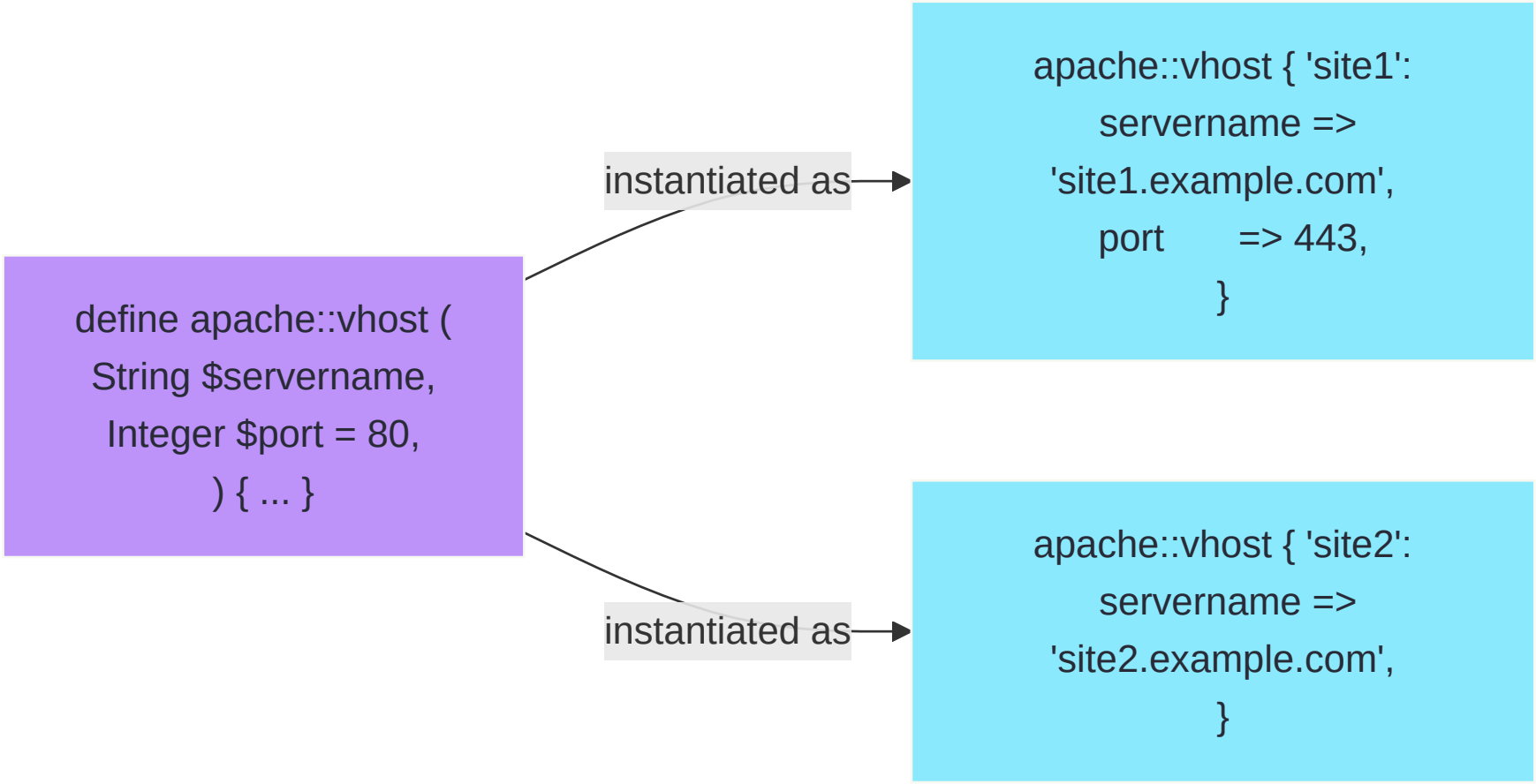
Using a Defined Type

```
# Multiple instances with different titles
vhost { 'my-site':
  servername    => 'www.example.com',
  document_root => '/var/www/production',
  port          => 443,
}

vhost { 'staging-site':
  servername    => 'staging.example.com',
  port          => 8080,
}

vhost { 'default-site':
  servername => 'localhost',
}
```

`define` vs. `class` — When to Use Which



class apache {
package { 'apache2': ... }

`define` vs. `class` — Decision Guide

	`class`	`define`
Instantiable	Once per node	Multiple times
Resource title	Class name	Any unique string
Use case	Managing a service (nginx, sshd)	Managing multiple instances (vhosts, users, cron jobs)
`include` **safe?**	Yes	N/A — use resource syntax

Part 4 — More Resource Types, Debugging & Operations

The `cron` Resource

Manage cron jobs declaratively — Puppet creates, updates, and removes cron entries:

```
# Run a backup every night at 2:00 AM
cron { 'nightly-backup':
  ensure  => present,
  command => '/opt/backup/run.sh >> /var/log/backup.log 2>&1',
  user    => 'root',
  hour    => 2,
  minute  => 0,
}

# Run every 15 minutes
cron { 'cache-warm':
  command => '/opt/app/warm-cache.sh',
  user    => 'www-data',
  minute  => ['0', '15', '30', '45'],
}

# Remove a stale cron job
cron { 'legacy-cleanup':
  ensure => absent,
```

The `notify` Resource — In-Catalog Debugging

```
# Print a message to the agent log during a run
notify { 'debug-port':
  message => "listen_port resolved to: ${listen_port}",
}

# Verify a value before it is applied
notify { 'pre-config-check':
  message => "worker_processes = ${worker_processes}",
  before  => File['/etc/nginx/nginx.conf'],
}

# Lightweight inline debug — no resource needed
notice("Current facts[os][family]: ${facts['os']['family']}")
```

> Remove `notify` resources and `notice()` calls before merging to production.
> Lint check: `puppet-lint --only-checks notice_and_warning`

The `tidy` Resource — Clean Up Stale Files

```
# Remove log files older than 30 days
tidy { '/var/log/myapp':
  age     => '30d',
  recurse => true,
  rmdirs  => false,
  matches => ['*.log'],
}

# Remove cache objects larger than 100 MB
tidy { '/opt/cache':
  size    => '100m',
  recurse => 1,
}
```

> **Age suffixes:** `s` seconds · `m` minutes · `h` hours · `d` days · `w` weeks
> Always test with `--noop` first — `tidy` is **not reversible**.

Resource Defaults

Set attribute defaults for every resource of a type within a scope:

```
class mymodule {
  # All File resources in this class inherit these defaults
  File {
    owner => 'root',
    group => 'root',
    mode  => '0644',
  }

  file { '/etc/myapp/app.conf':
    ensure  => file,
    content => epp('mymodule/app.conf.epp'),
    # owner, group, mode inherited
  }

  file { '/etc/myapp/secret.conf':
    ensure  => file,
    content => epp('mymodule/secret.conf.epp'),
    mode    => '0600',   # override for this one file only
  }
```

Virtual Resources

Declare resources that are **not applied** until explicitly realised:

```
# Declare (virtual — held in reserve)
@user { 'alice':
  ensure => present,
  uid    => 2001,
  groups => ['developers'],
}

@user { 'bob':
  ensure => present,
  uid    => 2002,
  groups => ['developers', 'ops'],
}

# Realise only ops-team users on ops nodes
User <| groups == 'ops' |>

# Or realise a specific named user
realize(User['alice'])
```

Virtual Resources — `exported` for Multi-Node

Exported resources share data between nodes via PuppetDB:

```
# On every web server — export its own nagios check
@@nagios_service { "check_http_${facts['networking']['fqdn']}":
  check_command       => 'check_http',
  host_name           => $facts['networking']['fqdn'],
  service_description => 'HTTP',
  target              => '/etc/nagios/conf.d/http_checks.cfg',
}

# On the Nagios server — collect all exported checks
Nagios_service <<| |>>
```

> Exported resources require **PuppetDB**. The `@@` prefix exports; `<<| |>>` collects. Essential for building dynamic configs like haproxy backends, Nagios checks, and SSH known_hosts.

The `puppet resource` Command

Inspect the live state of any resource — or write resources directly:

```
# Show the current state of all packages
puppet resource package

# Show a specific service in Puppet manifest syntax
puppet resource service nginx
# service { 'nginx':
#   ensure => 'running',
#   enable => 'true',
# }

# Show a specific file
puppet resource file /etc/ssh/sshd_config

# Apply a resource directly (bypasses catalog — use with care)
puppet resource package vim ensure=installed
```

> `puppet resource` is the fastest way to **onboard existing unmanaged servers**

`puppet apply` — Masterless Mode

Apply a manifest locally with no Puppet Server required:

```
# Apply a manifest file
puppet apply site.pp

# Dry-run — show what would change, apply nothing
puppet apply site.pp --noop

# Apply verbose
puppet apply site.pp --verbose --debug

# One-liner inline
puppet apply -e "package { 'git': ensure => installed }"

# With an explicit module path
puppet apply manifests/site.pp \
  --modulepath=./modules:$(puppet config print basemodulepath)
```


Use cases: development VMs, bootstrapping the Puppet Server itself, air-gapped

Node Classification in `site.pp`

`site.pp` maps nodes to roles using exact names, regex, or a catch-all default:

```
# Exact hostname
node 'web01.example.com' {
  include role::webserver
}

# Regex — matches web01, web02, web10 …
node /^web\d+\.example\.com$/ {
  include role::webserver
}

# Multiple hostnames in one block
node 'db01.example.com', 'db02.example.com' {
  include role::database
}

# Default — matches anything not explicitly listed
node default {
  include role::base
```

External Node Classifiers (ENC)

An ENC is any executable Puppet calls with the agent's certname; it returns YAML:

```
# puppet.conf — enable ENC
[server]
node_terminus  = exec
external_nodes = /usr/local/bin/my-enc.sh
```

```yaml
# stdout expected from the ENC
---
classes:
  role::webserver:
  profile::monitoring:
    prometheus_endpoint: 'prom.dc1.example.com'
parameters:
  datacenter: dc1
  tier: production
environment: production
```

Troubleshooting — Reading Agent Output

When `puppet agent --test` fails, the error points to the root cause:

```
Error: Could not retrieve catalog from remote server:
  Error 400 on SERVER: Evaluation Error: Duplicate declaration:
  Package[nginx] is already declared at manifests/init.pp:5
→ Fix: A class is being included twice via different code paths.
  Use 'include' (idempotent) — never 'class {}' more than once per node.
```

```
Error: Could not apply complete catalog: Could not find class ntp
  for agent01.example.com on node agent01.example.com
→ Fix: Module not installed / not on modulepath.
  Run: r10k deploy environment production -pv
  Check: puppet config print modulepath
```

```
Warning: Could not retrieve fact 'datacenter' for agent01
→ Fix: Custom fact script has an error.
  Debug: puppet facts --debug 2>&1 | grep datacenter
```

Common Agent Errors — Quick Reference

Error	Likely cause	Solution
`Duplicate declaration`	Resource declared twice	Use `include`; check module paths
`Could not find class X`	Module not on modulepath	Run `r10k deploy environment`
`Dependency cycle`	Circular `require` / `before`	Use `--graph` to visualise
`Certificate mismatch`	Certname ≠ signed cert name	`puppet ssl clean` + re-sign
`Connection refused :8140`	Puppet Server down	`systemctl start puppetserver`

Debugging with `--graph`

Puppet can export its dependency graph as `.dot` files for visual inspection:

```
# Generate during a test run
puppet agent --test --graph

# Or with apply
puppet apply manifests/site.pp --graph

# Graph files are written to:
ls /opt/puppetlabs/puppet/cache/state/graphs/
# expanded_relationships.dot
# relationships.dot
# resources.dot

# Convert to SVG with GraphViz
dot -Tsvg relationships.dot -o /tmp/puppet-graph.svg
```

> When you see **"Found dependency cycle"**, render the relationships graph — the

Puppet Server — Key Configuration Files

```
/etc/puppetlabs/puppet/
├── puppet.conf            ← main config (agent + server settings)
├── hiera.yaml             ← Hiera hierarchy (global layer)
└── ssl/                   ← CA, certificates, private keys

/etc/puppetlabs/puppetserver/
├── conf.d/
│   ├── puppetserver.conf ← JRuby pool size, timeouts
│   ├── webserver.conf    ← HTTPS port, SSL cert paths
│   └── auth.conf         ← API endpoint access control
└── services.d/
    └── ca.cfg            ← Enable/disable the CA service
```

```
# Reload server config without a full restart
systemctl reload puppetserver

# Print effective setting
puppetserver config print jruby-puppet.max-active-instances
```

Tuning the Puppet Agent

```
# /etc/puppetlabs/puppet/puppet.conf — [agent] section
[agent]
server              = puppet.example.com
environment         = production
runinterval         = 1800      # every 30 minutes
splaytime           = 300       # random start delay — stagger load on server
usecacheonfailure   = true      # use last good catalog if server unreachable
report              = true      # send run reports to PuppetDB
http_read_timeout   = 600       # 10 min — increase for large catalogs
```

```
# Check effective configuration
puppet config print --section agent

# Run immediately, skip splay delay
puppet agent --test --no-splay

# Safe dry run — see changes, apply nothing
puppet agent --test --noop
```

Tuning the Puppet Server

```
# /etc/sysconfig/puppetserver  (RHEL) or /etc/default/puppetserver (Debian)
JAVA_ARGS="-Xms4g -Xmx4g -XX:ReservedCodeCacheSize=512m"
```

```
# /etc/puppetlabs/puppetserver/conf.d/puppetserver.conf
jruby-puppet: {
    max-active-instances: 4           # roughly 1 per CPU core
    max-requests-per-instance: 10000  # recycle JRuby instance periodically
    compile-mode: jit
}
```

Concurrent agents	CPU cores	JVM heap	JRuby instances
≤ 20	2	2 GB	2
≤ 100	4	4 GB	4
≤ 500	8	8 GB	8

Day 1 — Summary

Concept	Key takeaway
Configuration drift	Puppet converges nodes to desired state on every run
AIO package	Bundled Ruby + Facter + Puppet — install once, everywhere
Certificates	Mutual TLS — every agent gets a signed cert from the server CA
Resources	The atomic unit of Puppet — type + title + attributes
Ordering	`require` / `before` order; `notify` / `subscribe` order + refresh
Conditionals	`if` / `unless` / `case` /selector — branch on facts or variables
Classes	Named, reusable blocks; include once per node
Defined types	Re-instantiable resource groups with unique titles

Day 1 — What's Next

Tomorrow we cover the powerful **extensions** of the Puppet language:

- Iterators and lambdas: `each`, `map`, `filter`, `reduce`
- Facts: built-in and custom (Ruby + external)
- Templates: ERB and EPP — generate config files from data
- Hiera: separate code from data, manage environments and exceptions elegantly

> Make sure to complete today's exercises before tomorrow — they build on each other.